Alternative Assessment Tool (AAT) Report on

"Design and Verification of Combinational and Sequential Digital Modules using Verilog HDL"

Submitted in partial fulfilment of the requirements of the Alternative Assessment Tool (AAT) for the course

ADVANCED DIGITAL AND LOGIC DESIGN (ADLD)

[Course Code: 23ECSPE1AD]

Submitted by

AKSHAY KUMAR 1BM25EC403

Submitted to

Prof. Ajaykumar Devarapalli

DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING

BMS College of Engineering
Bull Temple Road, Basavanagudi, Bengaluru-560019

Academic Year

2026 – 2027

1. Arithmetic Logic Unit (ALU)

The Arithmetic Logic Unit (ALU) is a combinational digital circuit that performs arithmetic and logical operations on binary data. The Verilog design uses two 4-bit inputs, A and B, a 3-bit selection input sel, and a 4-bit output Y. Based on the selection input, the ALU performs eight operations: addition, subtraction, AND, OR, XOR, NOT, left shift, and right shift. The case statement selects the required operation, and the output changes according to the selected operation. The testbench applies A = 1010 and B = 0011 and tests all eight operations by changing the selection input.

Design Code Testbench Code:

```
`timescale 1ns/1ps

module alu_tb;

reg  [3:0] A;
reg  [3:0] B;
reg  [2:0] sel;
wire [3:0] Y;

alu uut (
    .A(A),
    .B(B),
    .sel(sel),
    .Y(Y)
);

initial begin
    $dumpfile("alu.vcd");
    $dumpvars(0, alu_tb);

    $display("A    B    sel   Y");
    $monitor("%b  %b  %b  %b", A, B, sel, Y);

    A = 4'b0101;
    B = 4'b0011;

    sel = 3'b000; #10;  // Addition
    sel = 3'b001; #10;  // Subtraction
    sel = 3'b010; #10;  // AND
    sel = 3'b011; #10;  // OR
    sel = 3'b100; #10;  // XOR
    sel = 3'b101; #10;  // NOT
    sel = 3'b110; #10;  // Left shift
    sel = 3'b111; #10;  // Right shift

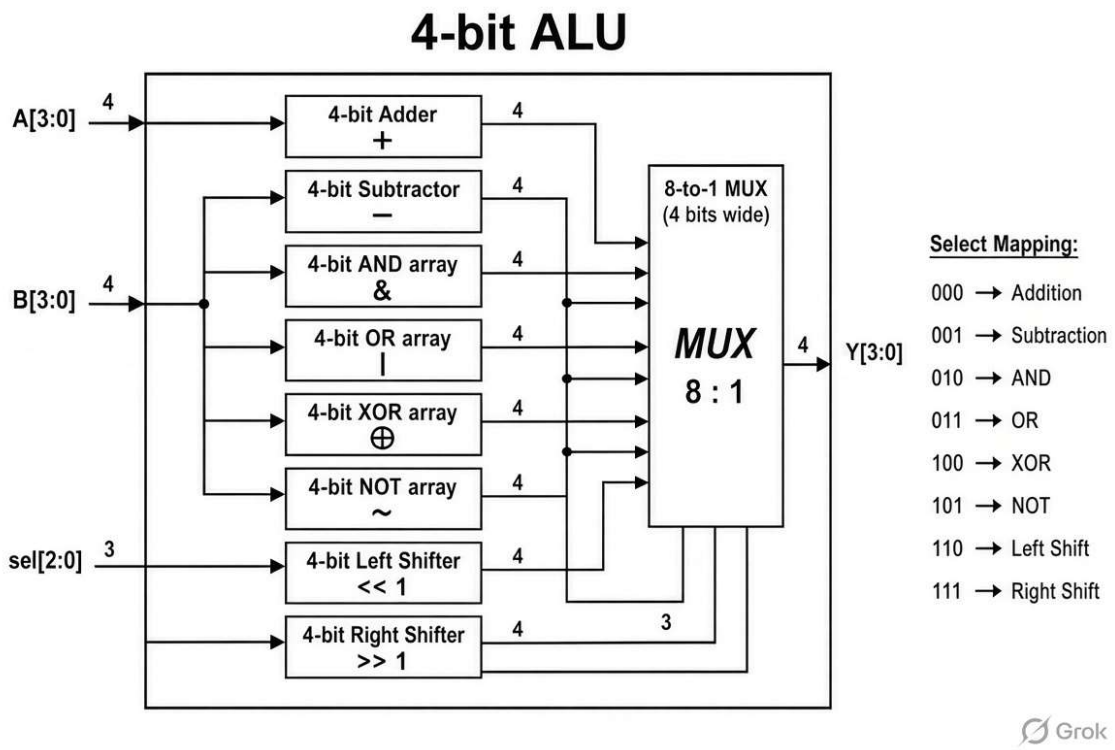
    $finish;
end
```

```
module alu (
    input  [3:0] A,
    input  [3:0] B,
    input  [2:0] sel,
    output reg [3:0] Y
);

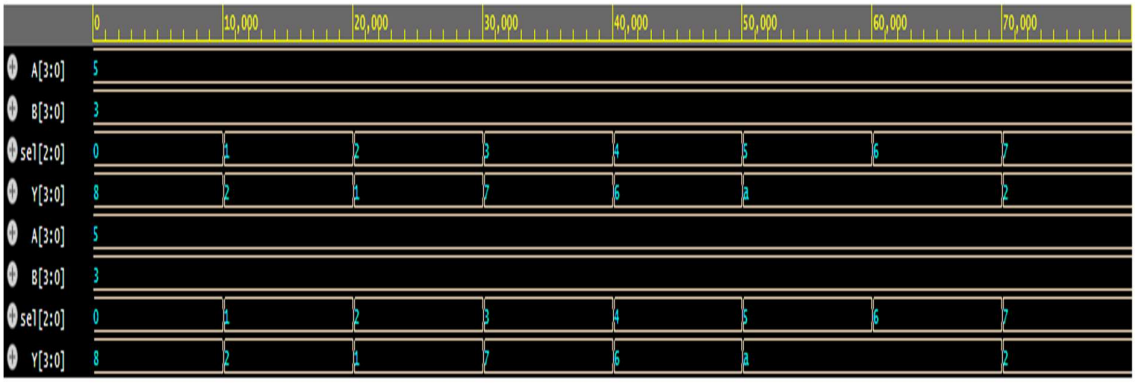
always @(*) begin
    case (sel)
        3'b000: Y = A + B;    // Addition
        3'b001: Y = A - B;    // Subtraction
        3'b010: Y = A & B;    // AND
        3'b011: Y = A | B;    // OR
        3'b100: Y = A ^ B;    // XOR
        3'b101: Y = ~A;       // NOT
        3'b110: Y = A << 1;   // Left shift
        3'b111: Y = A >> 1;   // Right shift
        default: Y = 4'b0000;
    endcase
end

endmodule
```

Circuit Diagrams:



Output Waveform:



Output Waveform Explanation:

Initially, the inputs are set to A = 1010 and B = 0011. The testbench changes the selection input sel at intervals of 10 ns to verify each ALU operation. When sel = 000, addition is performed; 001 performs subtraction; 010 performs bitwise AND; 011 performs bitwise OR; 100 performs bitwise XOR; 101 performs bitwise NOT of A; 110 shifts A left by one bit; and 111 shifts A right by one bit. The output Y changes immediately according to the selected operation because the ALU is a combinational circuit

2. N-BIT COMPARATOR

A comparator is a combinational logic circuit used to compare two binary numbers. The N-bit comparator has two N-bit inputs, A and B, and three outputs: GT, LT, and EQ. The parameter N allows the width of the inputs to be changed.

Design Code Testbench Code:

```
module tb;
parameter N = 4;
reg  [N-1:0] A, B;
wire GT, LT, EQ;
n_bit_comparator #(N) dut (
    .A(A),
    .B(B),
    .GT(GT),
    .LT(LT),
    .EQ(EQ)
);
initial begin
    $dumpfile("dump.vcd");
    $dumpvars(0, tb);

    $monitor("Time=%0t A=%b B=%b GT=%b LT=%b EQ=%b",
             $time, A, B, GT, LT, EQ);

    A = 4'b1010; B = 4'b0101; #10;
    A = 4'b0011; B = 4'b1001; #10;
    A = 4'b0110; B = 4'b0110; #10;
    A = 4'b1111; B = 4'b0001; #10;
    A = 4'b0000; B = 4'b1111; #10;

    $finish;
end
endmodule
```

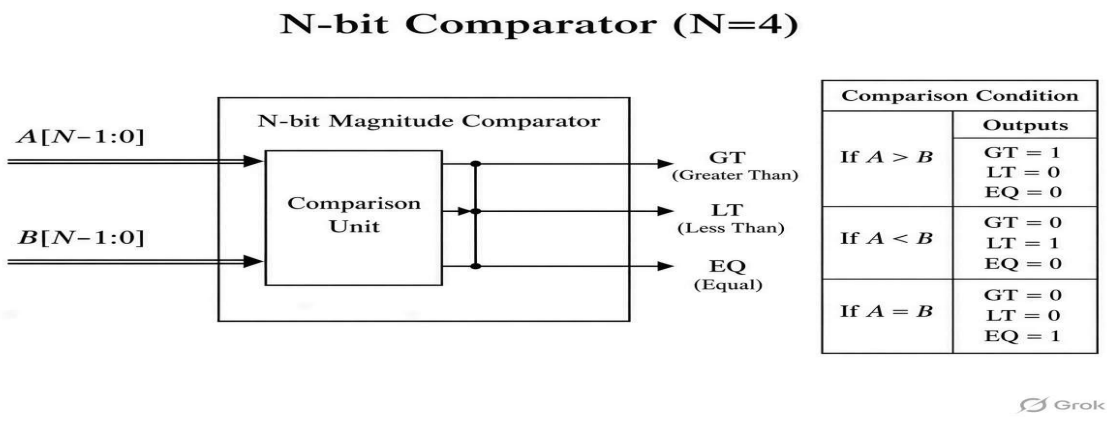
```
module n_bit_comparator #(parameter N = 4) (
    input  [N-1:0] A,
    input  [N-1:0] B,
    output reg GT,
    output reg LT,
    output reg EQ
);

always @(*) begin
    if (A > B) begin
        GT = 1'b1;
        LT = 1'b0;
        EQ = 1'b0;
    end
    else if (A < B) begin
        GT = 1'b0;
        LT = 1'b1;
        EQ = 1'b0;
    end
    else begin
        GT = 1'b0;
        LT = 1'b0;
        EQ = 1'b1;
    end
end

endmodule

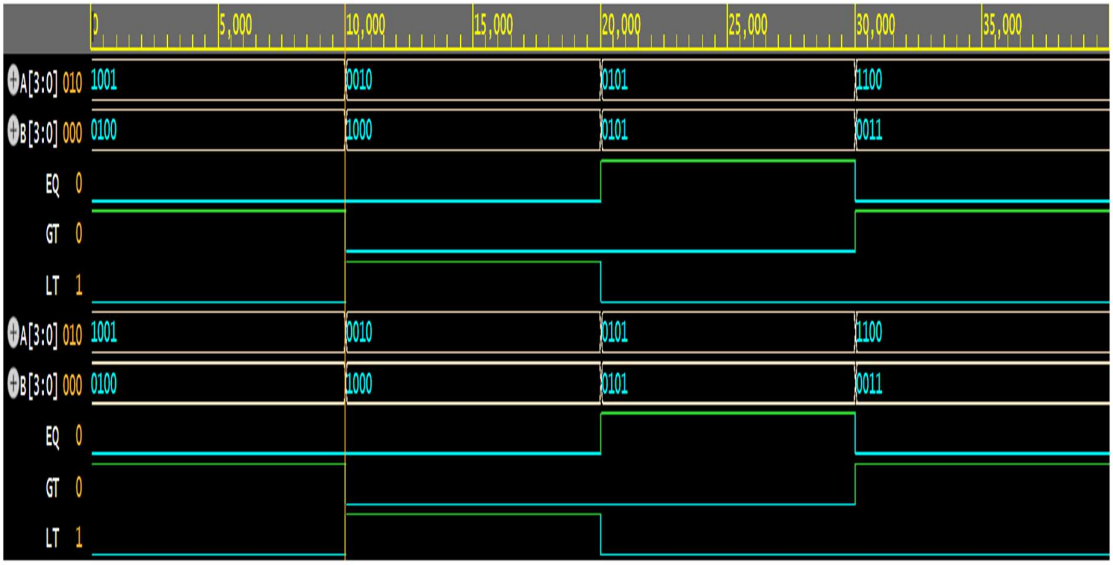
```

Circuit Diagrams:



Grok

Output Waveform:



Output Waveform Explanation:

The waveform displays the input signals A and B and the output signals GT, LT, and EQ. When A is greater than B, GT becomes high. When A is less than B, LT becomes high. When both inputs are equal, EQ becomes high. For each comparison, only one output should be high.

3. 4-Bit Parity Generator and Checker

The parity generator calculates the parity bit using the XOR operation on the 4-bit input data. The parity checker compares the calculated parity with the received parity_in bit. If both values match, error = 0; otherwise, error = 1.

Design Code Testbench Code:

```
module tb;
    reg [3:0] data;
    reg parity_in;
    wire parity_out;
    wire error;
    parity uut (
        .data(data),
        .parity_in(parity_in),
        .parity_out(parity_out),
        .error(error)
    );
    initial begin
        $dumpfile("parity.vcd");
        $dumpvars(0, tb);
        data = 4'b0000; parity_in = 0;
        #10;data = 4'b0001; parity_in = 1;
        #10;
        data = 4'b0011; parity_in = 0;
        #10;data = 4'b1010; parity_in = 0;
        #10;
        data = 4'b1010; parity_in = 1;
        #10;
        $finish;
    end
    initial begin
        $monitor("Time=%0t data=%b parity_in=%b
parity_out=%b error=%b",
                 $time, data, parity_in, parity_out,
error);
    end
endmodule
```

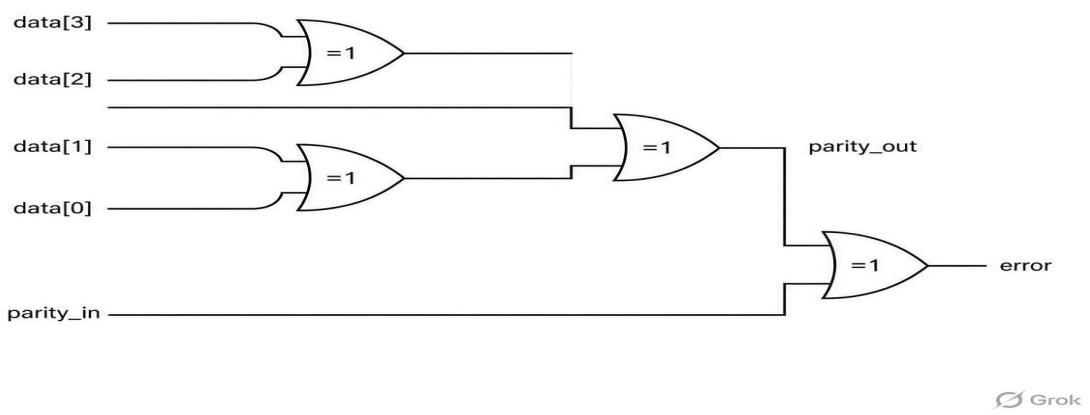
```
module parity (
    input [3:0] data,
    input parity_in,
    output reg parity_out,
    output reg error
);

    always @(*) begin
        // Even parity generator
        parity_out = ^data;

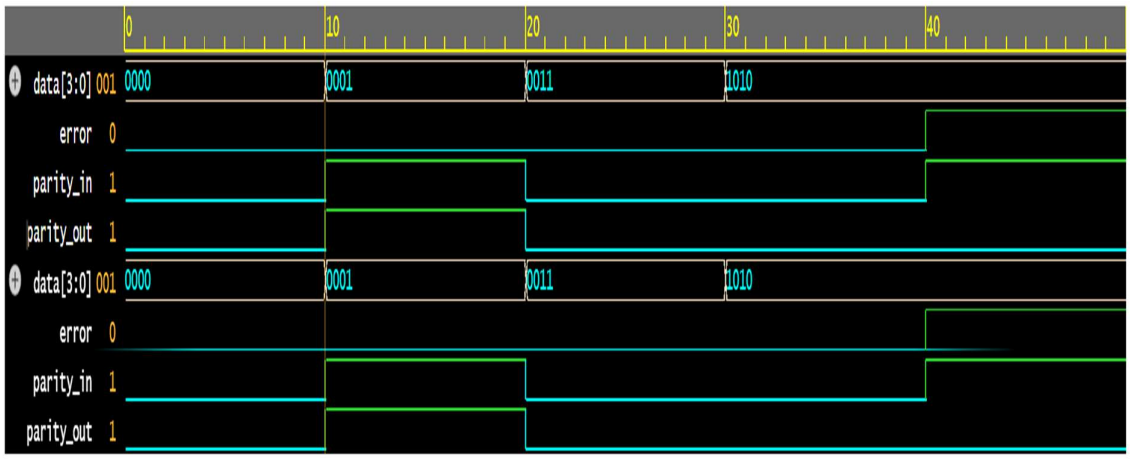
        // Parity checker
        if ((^data) == parity_in)
            error = 1'b0;
        else
            error = 1'b1;
    end

endmodule
```

Circuit Diagrams:



Output Waveform:



Output Waveform Explanation:

The waveform shows the input signals data and parity_in, along with the output signals parity_out and error. For data = 0000 and parity_in = 0, the error output is 0. For data = 0001 and parity_in = 1, the error output is also 0. For data = 0011 and parity_in = 0, the error output is 0. For data = 1010 and parity_in = 0, the error output is 0. Finally, for data = 1010 and parity_in = 1, the error output is 1.

4. 4-Bit Barrel Shifter

A barrel shifter is a combinational circuit that shifts or rotates binary data by a specified number of bit positions. In this experiment, a 4-bit circular right-rotation operation is performed based on the 2-bit select input sel.

Design Code Testbench Code:

```
module tb;

reg [3:0] w;
reg [1:0] sel;
wire [3:0] y;

barrel_shifter dut (
    .w(w),
    .sel(sel),
    .y(y)
);

initial begin
    $dumpfile("dump.vcd");
    $dumpvars(0, tb);

    w = 4'b1010; sel = 2'b00; #10;
    w = 4'b1010; sel = 2'b01; #10;
    w = 4'b1010; sel = 2'b10; #10;
    w = 4'b1010; sel = 2'b11; #10;

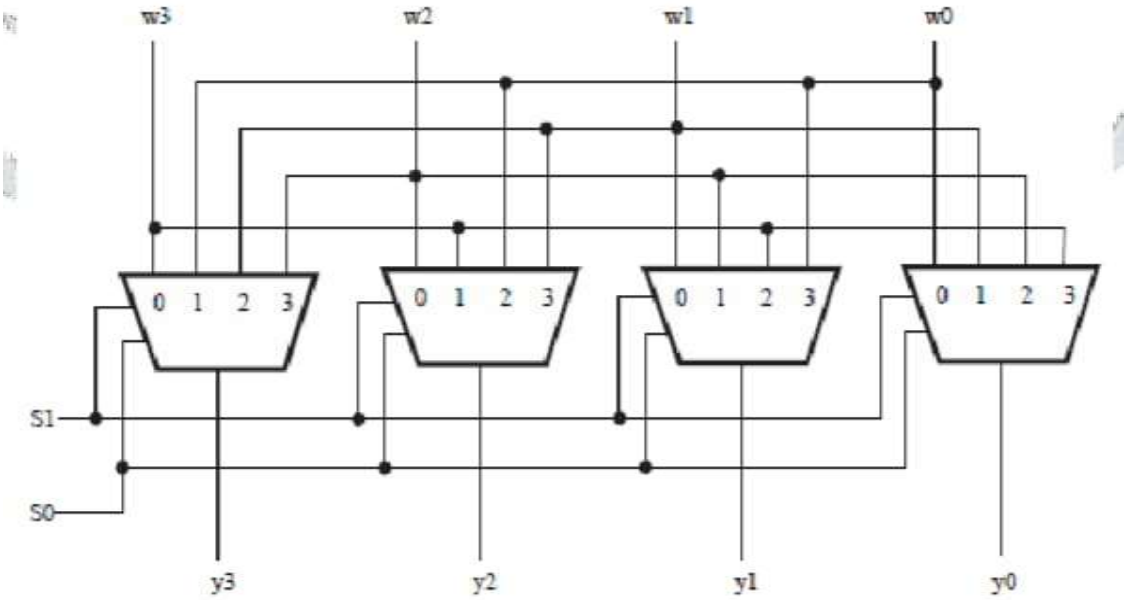
    w = 4'b1101; sel = 2'b00; #10;
    w = 4'b1101; sel = 2'b01; #10;
    w = 4'b1101; sel = 2'b10; #10;
    w = 4'b1101; sel = 2'b11; #10;
```

```
module barrel_shifter (
    input [3:0] w,
    input [1:0] sel,
    output reg [3:0] y
);

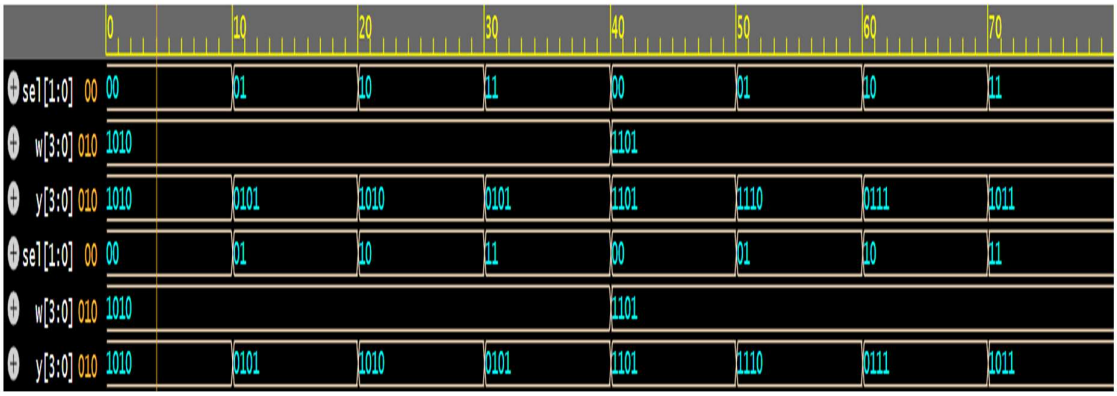
always @(*) begin
    case (sel)
        2'b00: y = w;
        2'b01: y = {w[0], w[3], w[2], w[1]};
        2'b10: y = {w[1], w[0], w[3], w[2]};
        2'b11: y = {w[2], w[1], w[0], w[3]};
    endcase
end

endmodule
```

Circuit Diagrams:



Output Waveform:



Output Waveform Explanation:

The waveform shows the input data w, select input sel, and output y. When sel = 00, the output remains unchanged. When sel = 01, the data rotates right by one bit. When sel = 10, it rotates right by two bits. When sel = 11, it rotates right by three bits. The output changes according to the select input.

5. Ring Counter

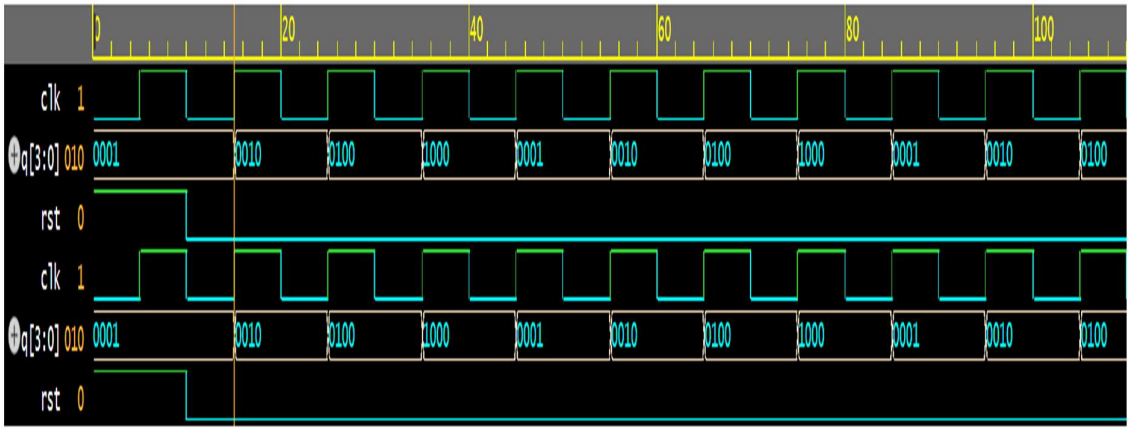
A 4-bit ring counter is a sequential circuit in which the output of the last flip-flop is fed back to the first flip-flop. The counter uses a clock signal to shift a single logic 1 through the four bits. The reset initializes the output to 0001.

Design Code Testbench Code:

```
module tb;
    reg clk, rst;
    wire [3:0] q;
    ring_counter uut (
        .clk(clk),
        .rst(rst),
        .q(q)
    );
    initial begin
        $dumpfile("ring_counter.vcd");
        $dumpvars(0, tb);
        clk = 0;
        rst = 1;
        #10;
        rst = 0;
        #100;
        $finish;
    end
    always #5 clk = ~clk;
    initial begin
        $monitor("Time=%0t clk=%b rst=%b q=%b",
                 $time, clk, rst, q);
    end
endmodule
```

```
module ring_counter (
    input clk,
    input rst,
    output reg [3:0] q
);
    always @(posedge clk or posedge rst) begin
        if (rst)
            q <= 4'b0001;
        else
            q <= {q[2:0], q[3]};
    end
endmodule
```

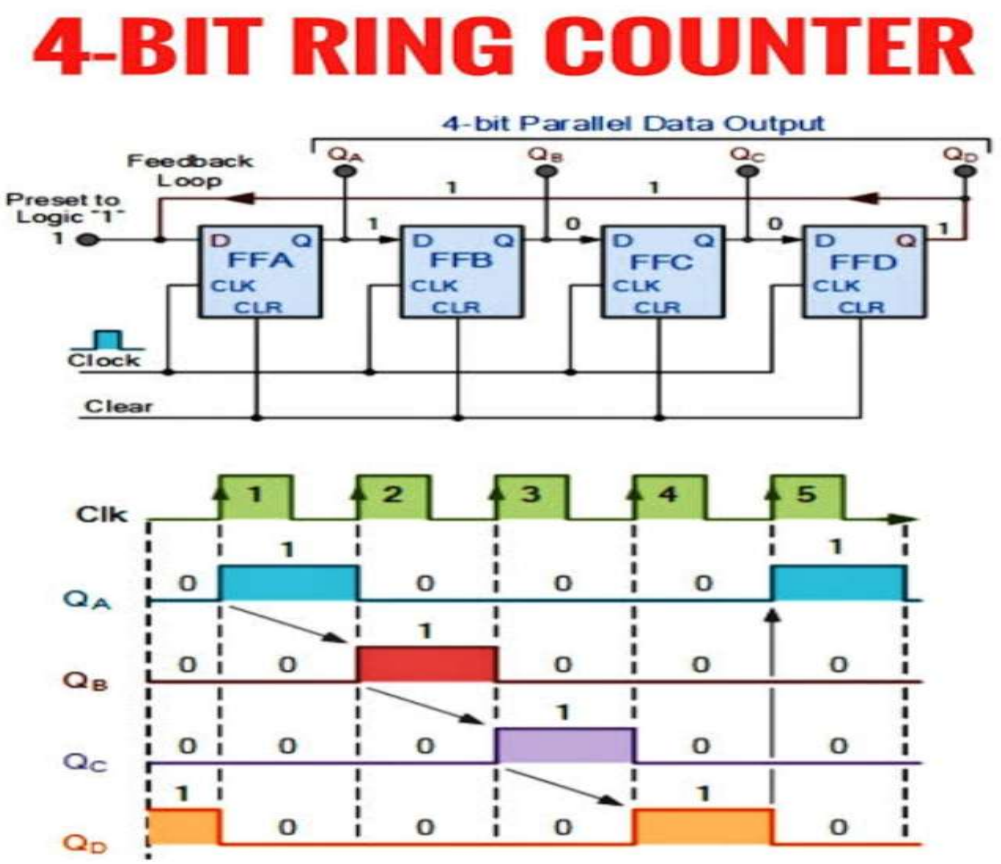
Output Waveform:



Output Waveform Explanation:

The waveform shows the clock, reset, and 4-bit output. After reset is released, the output shifts at every rising edge of the clock in the sequence 0001 → 0010 → 0100 → 1000 → 0001. This sequence repeats continuously, verifying the correct operation of the ring counter.

Circuit Diagrams:



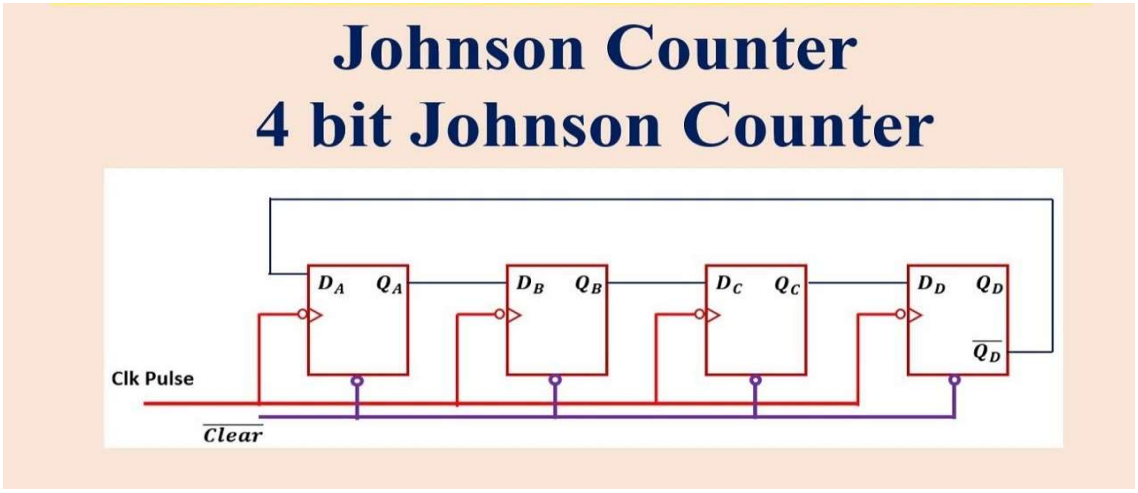
6. 4-Bit Johnson Counter

Design Code Testbench Code:

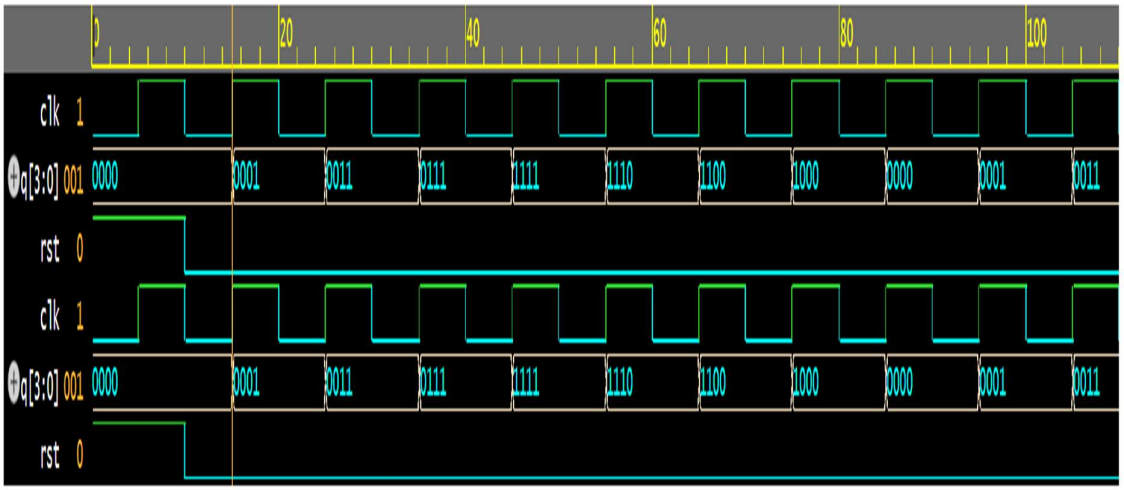
```
module tb;
    reg clk, rst;
    wire [3:0] q;
    johnson_counter uut (
        .clk(clk),
        .rst(rst),
        .q(q)
    );
    initial begin
        $dumpfile("johnson_counter.vcd");
        $dumpvars(0, tb);
        clk = 0;
        rst = 1;
        #10;
        rst = 0;
        #100;
        $finish;
    end
    always #5 clk = ~clk;
    initial begin
        $monitor("Time=%0t clk=%b rst=%b q=%b",
                 $time, clk, rst, q);
    end
endmodule
```

```
module johnson_counter (
    input clk,
    input rst,
    output reg [3:0] q
);
    always @(posedge clk or posedge rst) begin
        if (rst)
            q <= 4'b0000;
        else
            q <= {q[2:0], ~q[3]};
    end
endmodule
```

Circuit Diagrams:



Output Waveform:



Output Waveform Explanation:

The waveform shows the clock, reset, and 4-bit output. After reset is released, the output changes at each rising edge of the clock in the sequence:

0000 → 0001 → 0011 → 0111 → 1111 → 1110 → 1100 → 1000 → 0000

The sequence repeats continuously, verifying the correct operation of the 4-bit Johnson counter.

7. Synchronous and Asynchronous Counters

1. Synchronous Counter

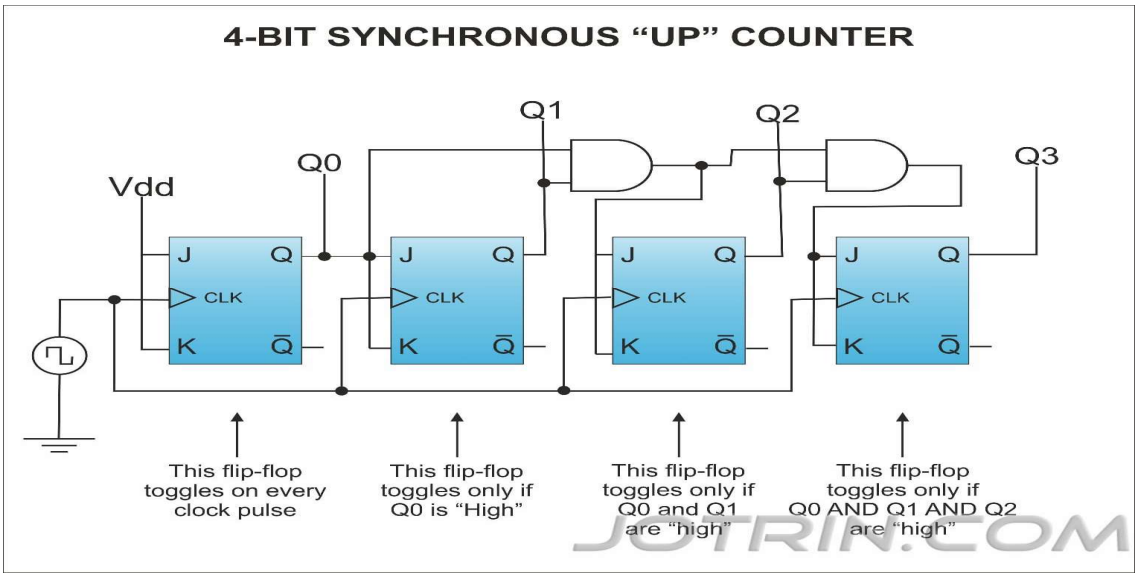
A synchronous counter is a sequential circuit in which all flip-flops receive the same clock signal. The counter increments its binary value at every rising edge of the clock. The reset initializes the output to 0000.

Design Code Testbench Code:

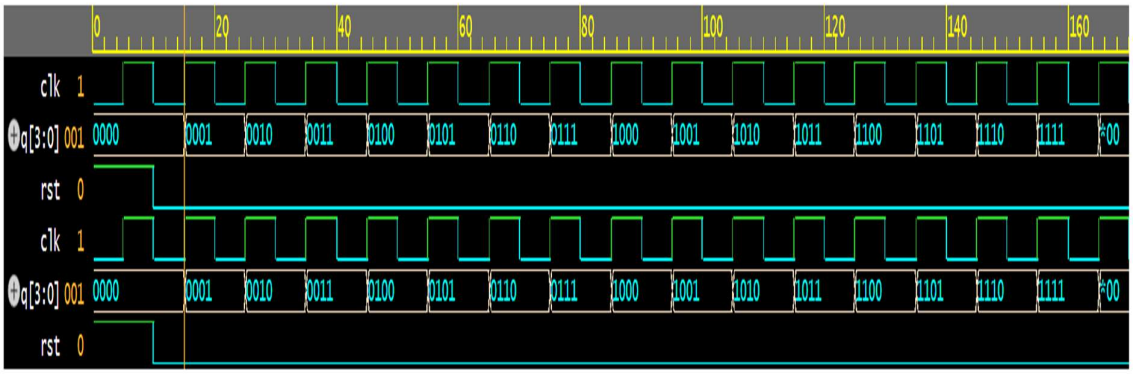
```
module tb;
    reg clk, rst;
    wire [3:0] q;
    sync_counter uut (
        .clk(clk),
        .rst(rst),
        .q(q)
    );
    initial begin
        $dumpfile("sync_counter.vcd");
        $dumpvars(0, tb);
        clk = 0;
        rst = 1;
        #10;
        rst = 0;
        #160;
        $finish;
    end
    always #5 clk = ~clk;
    initial begin
        $monitor("Time=%0t clk=%b rst=%b q=%b",
                 $time, clk, rst, q);
    end
endmodule
```

```
module sync_counter (
    input clk,
    input rst,
    output reg [3:0] q
);
    always @(posedge clk or posedge rst) begin
        if (rst)
            q <= 4'b0000;
        else
            q <= q + 1'b1;
    end
endmodule
```

Circuit Diagrams:



Output Waveform:



Output Waveform Explanation:

The waveform shows the clock, reset, and 4-bit output. After reset is released, the output increments from 0000 to 0001, 0010, 0011, and continues in binary order at every rising clock edge.

2. Asynchronous Counter

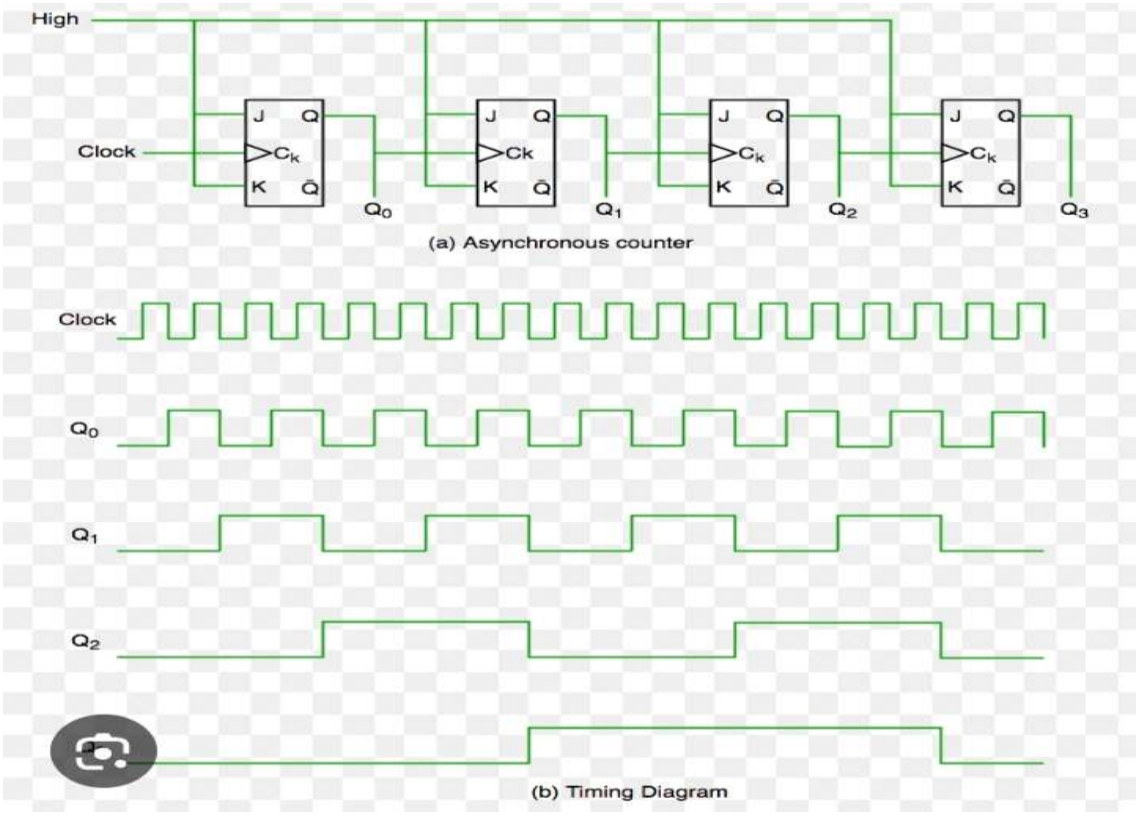
An asynchronous counter, also called a ripple counter, is a sequential circuit in which only the first flip-flop receives the external clock. Each subsequent flip-flop is triggered by the output transition of the previous flip-flop. Therefore, the changes ripple through the counter stages.

Design Code Testbench Code:

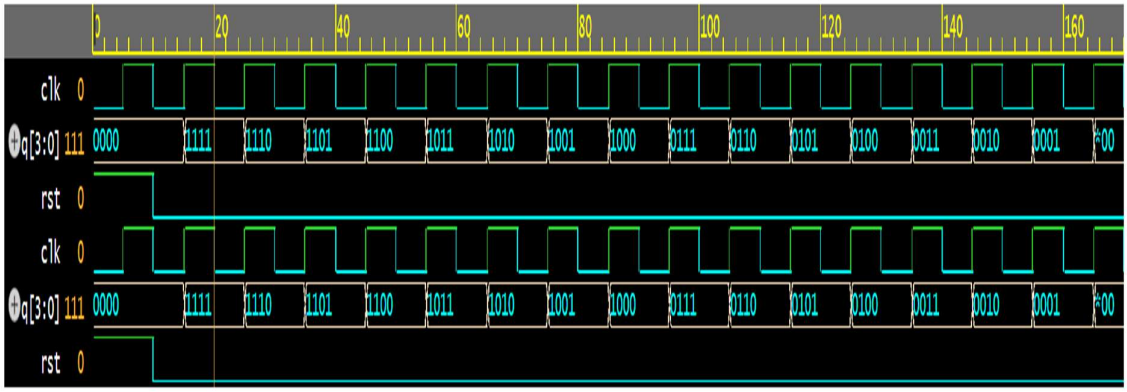
```
module tb;
    reg clk, rst;
    wire [3:0] q;
    async_counter uut (
        .clk(clk),
        .rst(rst),
        .q(q)
    );
    initial begin
        $dumpfile("async_counter.vcd");
        $dumpvars(0, tb);
        clk = 0;
        rst = 1;
        #10;
        rst = 0;
        #160;
        $finish;
    end
    always #5 clk = ~clk;
    initial begin
        $monitor("Time=%0t clk=%b rst=%b q=%b",
                 $time, clk, rst, q);
    end
endmodule
```

```
module async_counter (
    input clk,
    input rst,
    output reg [3:0] q
);
  always @(posedge clk or posedge rst) begin
        if (rst)
            q[0] <= 1'b0;
        else
            q[0] <= ~q[0];end
    always @(posedge q[0] or posedge rst) begin
        if (rst)
            q[1] <= 1'b0;
        else
            q[1] <= ~q[1]; end
    always @(posedge q[1] or posedge rst) begin
        if (rst)
            q[2] <= 1'b0;
        else
            q[2] <= ~q[2];
    end
    always @(posedge q[2] or posedge rst) begin
        if (rst)
            q[3] <= 1'b0;
       else
            q[3] <= ~q[3];
    end
endmodule
```

Circuit Diagrams:



Output Waveform:



Output Waveform Explanation

The waveform shows the clock, reset, and 4-bit output. After reset is released, the least significant bit toggles with the external clock, while the remaining bits toggle at progressively lower frequencies. The output follows the binary counting sequence from 0000 to 1111 and then repeats.

8. Divide-by-N counter

A Divide-by-N counter divides the input clock frequency by a specified factor. The counter counts clock pulses and toggles the output after a fixed number of pulses.

Design Code Testbench Code:

```
module tb;
 reg clk, rst;
    wire clk_out;
    divide_by_n uut (
        .clk(clk),
        .rst(rst),
        .clk_out(clk_out)
    );
  initial begin
        clk = 0;
        forever #5 clk = ~clk;
    end

    initial begin
        $dumpfile("divide_by_n.vcd");
        $dumpvars(0, tb);

        rst = 1;
        #10;

        rst = 0;
        #200;

        $finish;
    end
  initial begin
        $monitor("Time=%0t clk=%b rst=%b clk_out=%b",
                 $time, clk, rst, clk_out);
    end

endmodule
```

```

module divide_by_n (
    input clk,
    input rst,
    output reg clk_out
);

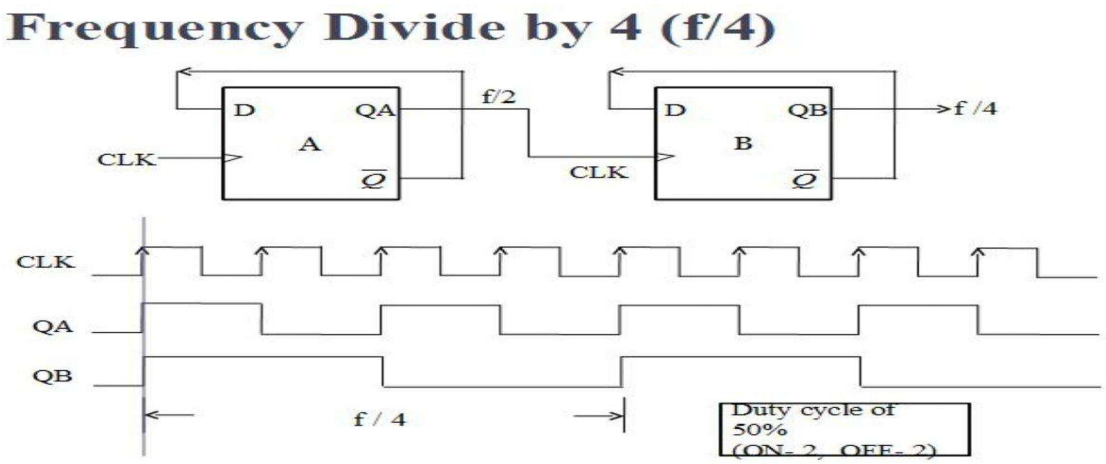
    parameter N = 4;
    integer count;

    always @(posedge clk or posedge rst) begin
        if (rst) begin
            count <= 0;
            clk_out <= 0;
        end
        else begin
            if (count == (N/2)-1) begin
                count <= 0;
                clk_out <= ~clk_out;
            end
            else
                count <= count + 1;
        end
    end

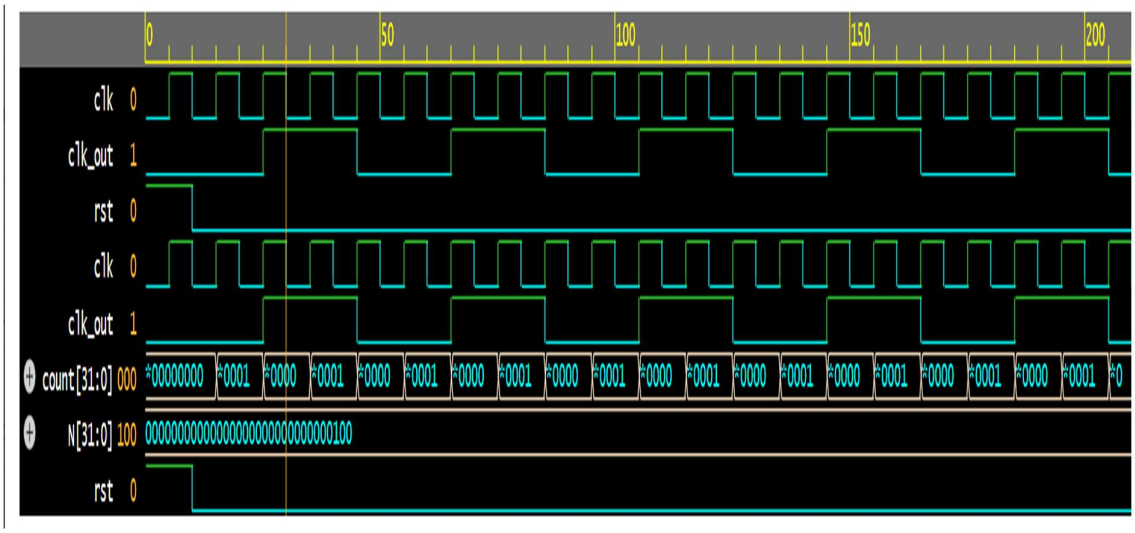
endmodule

```

Circuit Diagrams:



Output Waveform:



Output Explanation:

Initially, the reset signal sets the counter and output to zero. After reset is released, the counter counts the input clock pulses. The output toggles after every two clock pulses, producing a frequency equal to one-fourth of the input clock frequency

9. Random Sequence Generator

random sequence generator generates a sequence of binary numbers. In digital circuits, a Linear Feedback Shift Register (LFSR) is commonly used to generate a pseudo-random sequence.

An LFSR consists of flip-flops connected in a shift-register arrangement and an XOR gate for feedback. At every positive edge of the clock, the register shifts its bits and calculates a new input bit using XOR logic.

Design Code Testbench Code:

```
module tb;
  reg clk, rst;
    wire [3:0] Q;
  random_sequence uut (
        .clk(clk),
        .rst(rst),
        .Q(Q)
    );
  initial begin
        clk = 0;
        forever #5 clk = ~clk;
    end
  initial begin
        $dumpfile("random_sequence.vcd");
        $dumpvars(0, tb);

        rst = 1;
        #10;

        rst = 0;
        #160;

        $finish;
    end

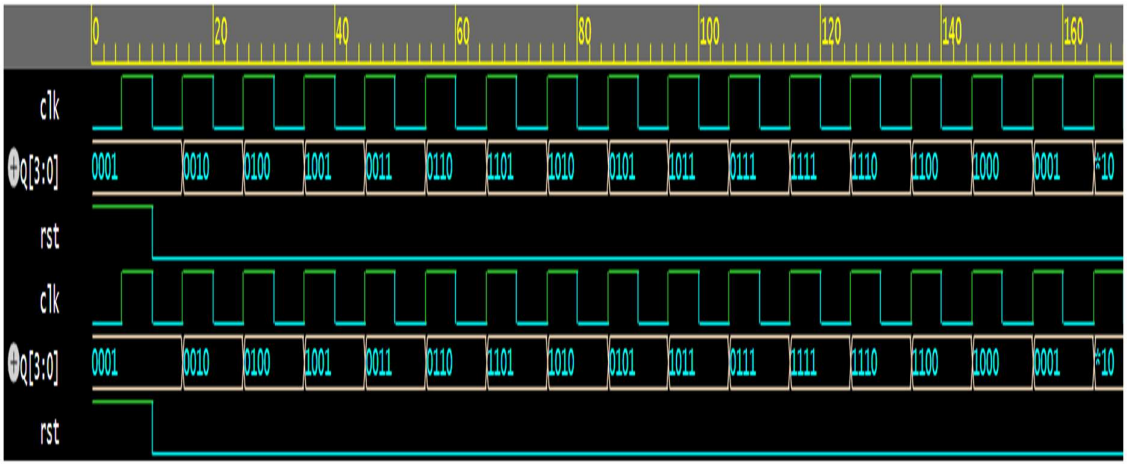
    initial begin
        $monitor("Time=%0t clk=%b rst=%b Q=%b",
                 $time, clk, rst, Q);
    end
endmodule
```

```
module random_sequence (
    input clk,
    input rst,
    output reg [3:0] Q
);

    always @(posedge clk or posedge rst) begin
        if (rst)
            Q <= 4'b0001;
        else
            Q <= {Q[2:0], Q[3] ^ Q[2]};
    end

endmodule
```

Output Waveform:



Output Explanation:

Initially, the reset signal is activated, and the output register is set to 0001.

After the reset is released, the circuit generates a new output at every rising edge of the clock. The XOR feedback logic calculates the next bit, while the remaining bits shift to their new positions.

he generated 4-bit sequence is:

0001 → 0011 → 0111 → 1111 → 1110 → 1101 → 1010 → 0101 → 1011 → 0110 → 1100 → 1001 → 0010 → 0100 → 1000 → 0001

10. Timer Module

The timer module uses a 4-bit counter to perform a countdown operation. The inputs are clk, rst, start, and load_value, while the outputs are count and done. When the reset signal (rst) is high, the counter and done signal are cleared. When start is high, the counter loads the value provided by load_value. The timer then decrements the counter by one on each positive edge of the clock. When the counter reaches 1, it resets the count to zero and sets done to high, indicating that the countdown is complete. The testbench generates the clock signal, applies reset, loads the value 5, starts the countdown, and monitors the output signals during simulation

Design Code Testbench Code:

```
`timescale 1ns/1ps

module timer_tb;

reg clk;
reg rst;
reg start;
reg [3:0] load_value;

wire [3:0] count;
wire done;

timer uut (
    .clk(clk),
    .rst(rst),
    .start(start),
    .load_value(load_value),
    .count(count),
    .done(done)
);

// Generate clock with 10 ns period
initial begin
    clk = 0;
    forever #5 clk = ~clk;
end

initial begin
    $dumpfile("timer.vcd");
    $dumpvars(0, timer_tb);

    $monitor("Time=%0t rst=%b start=%b count=%d done=%b",
             $time, rst, start, count, done);

    // Apply reset
    rst = 1;
    start = 0;
    load_value = 4'd5;

    @(negedge clk);
    rst = 0;

    // Start countdown from 5
    start = 1;

    @(negedge clk);
    start = 0;

    // Allow countdown to complete
    repeat (7) @(negedge clk);

    $finish;
end

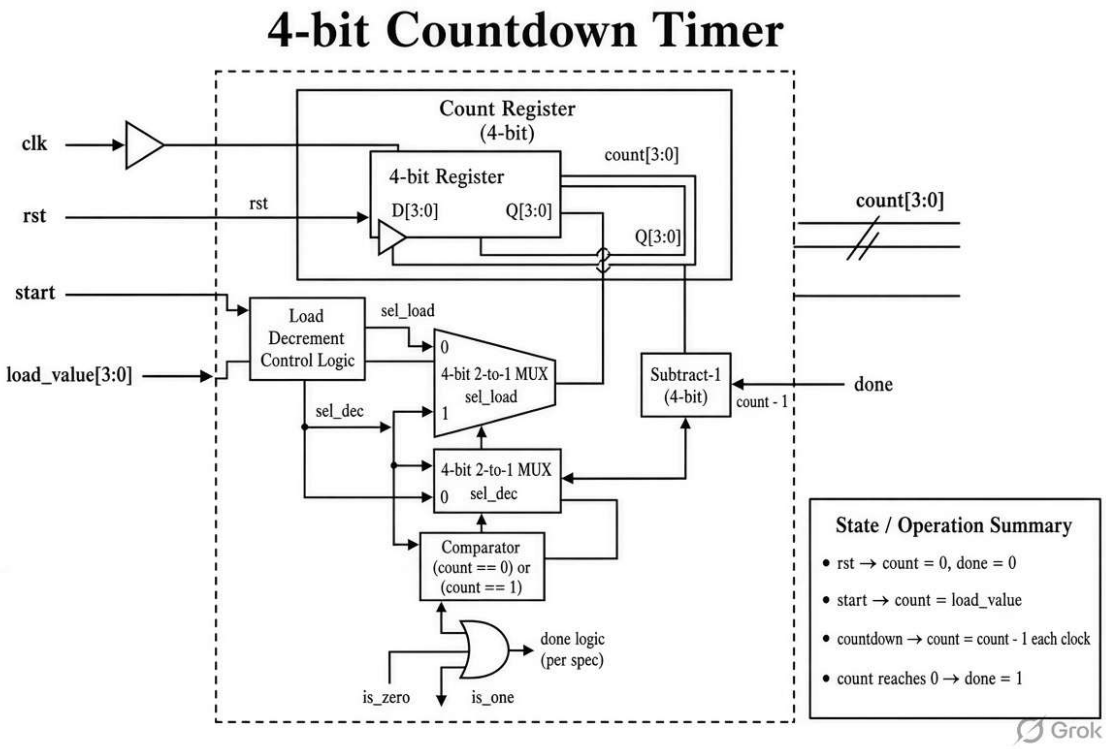
endmodule
```

```
module timer (
    input clk,
    input rst,
    input start,
    input [3:0] load_value,
    output reg [3:0] count,
    output reg done
);

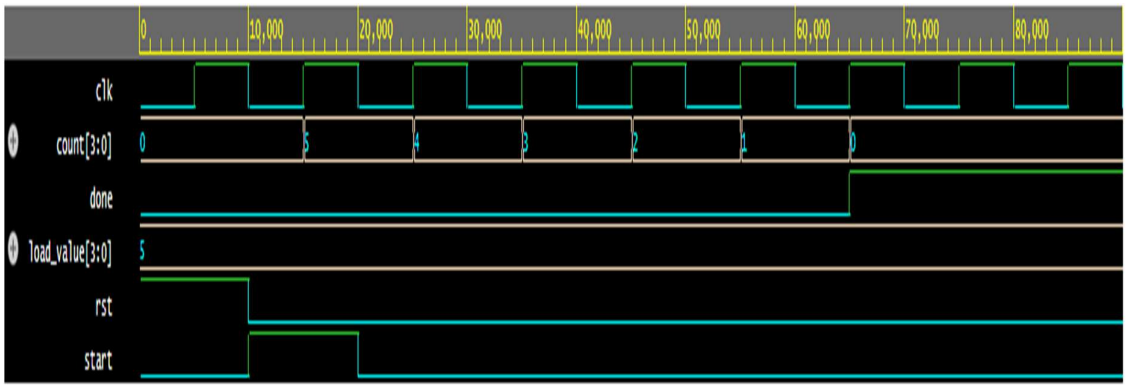
always @(posedge clk) begin
    if (rst) begin
        count <= 4'b0000;
        done  <= 1'b0;
    end
    else if (start) begin
        count <= load_value;
        done  <= (load_value == 0);
    end
    else if (count > 0) begin
        if (count == 1) begin
            count <= 4'b0000;
            done  <= 1'b1;
        end
        else begin
            count <= count - 1'b1;
        end
    end
end

endmodule
```

Circuit Diagrams:



Output Waveform:



Output Waveform Explanation:

Initially, the reset signal (rst) is set to 1, which clears the counter (count) and the done signal to 0. After reset is deactivated, the start signal becomes 1, and the counter loads the value 5 from load_value. When start returns to 0, the timer begins counting down on each positive edge of the clock: 5, 4, 3, 2, and 1. When the counter reaches 1, it resets to 0 and the done signal becomes 1, indicating that the countdown is complete. The waveform verifies the correct operation of the countdown timer.

11. Shift Register

A shift register is a sequential digital circuit used to store and shift binary data. The Verilog design uses a 4-bit register with a clock signal, reset signal, serial inputs (SR and SL), parallel input (D), and a 2-bit mode selector (mode). When the reset signal (rst) is high, the output register Q is cleared to 0000. Based on the selected mode, the register performs four operations: hold keeps the existing data unchanged, shift right shifts the bits to the right and inserts the serial input SR, shift left shifts the bits to the left and inserts the serial input SL, and parallel load loads all four input bits D into the register. These operations occur on the positive edge of the clock.

Design Code Testbench Code:

```
`timescale 1ns/1ps

module shift_register_tb;

reg clk;
reg rst;
reg [1:0] mode;
reg [3:0] D;
reg SR, SL;
wire [3:0] Q;

shift_register uut (
    .clk(clk),
    .rst(rst),
    .mode(mode),
    .D(D),
    .SR(SR),
    .SL(SL),
    .Q(Q)
);

// Generate clock with 10 ns period
initial begin
    clk = 0;
    forever #5 clk = ~clk;
end

initial begin
    $dumpfile("shift_register.vcd");
    $dumpvars(0, shift_register_tb);

    $monitor("Time=%0t rst=%b mode=%b D=%b SR=%b SL=%b Q=%b",
             $time, rst, mode, D, SR, SL, Q);

    // Reset
    rst = 1;
    mode = 2'b00;
```

```
module shift_register (
    input clk,
    input rst,
    input [1:0] mode,
    input [3:0] D,
    input SR,
    input SL,
    output reg [3:0] Q
);

always @(posedge clk) begin
    if (rst)
        Q <= 4'b0000;

    else begin
        case (mode)
            2'b00: Q <= Q;          // Hold
            2'b01: Q <= {SR, Q[3:1]}; // Shift right
            2'b10: Q <= {Q[2:0], SL}; // Shift left
            2'b11: Q <= D;           // Parallel load
            default: Q <= Q;
        endcase
    end
end

endmodule
```

```
        D = 4'b0000;
        SR = 0;
        SL = 0;
        #10;

        // Parallel load 1011
        rst = 0;
        mode = 2'b11;
        D = 4'b1011;
        #10;

        // Hold data
        mode = 2'b00;
        #10;

        // Shift right, insert 0
        mode = 2'b01;
        SR = 0;
        #10;

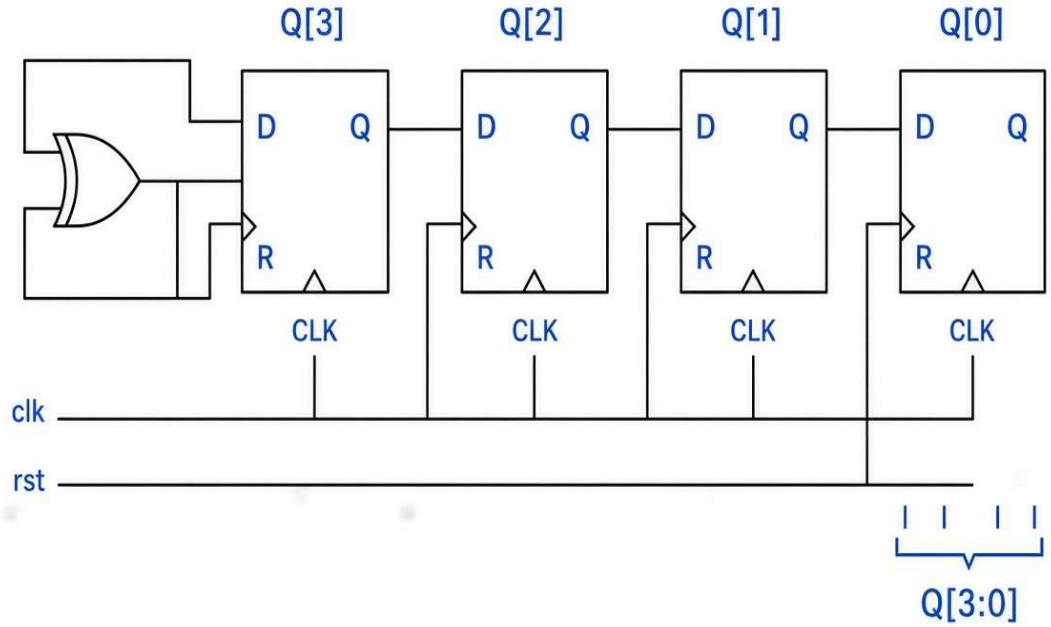
        // Shift left, insert 1
        mode = 2'b10;
        SL = 1;
        #10;

        // Parallel load 1100
        mode = 2'b11;
        D = 4'b1100;
        #10;

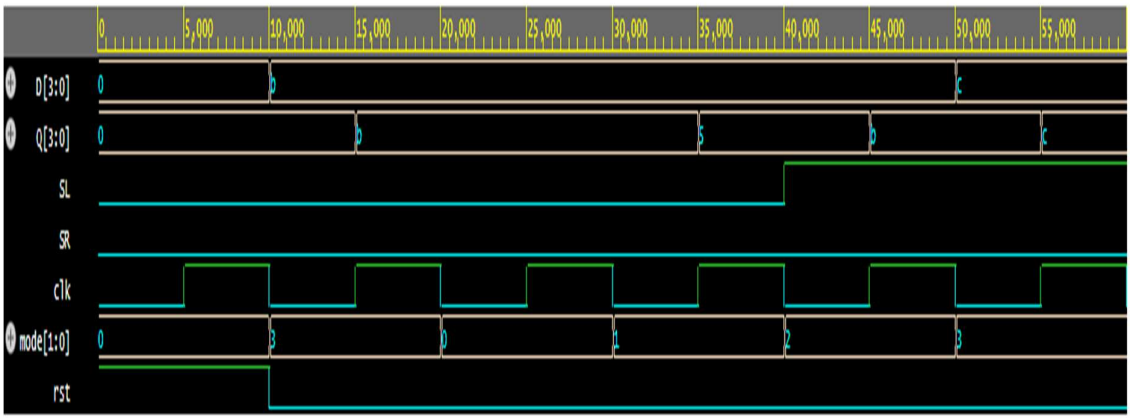
        $finish;
    end

endmodule
```

Circuit Diagrams:



Output Waveform:



Initially, the reset signal (rst) is set to 1, which clears the output register Q to 0000. After reset is deactivated, the parallel load mode (mode = 2'b11) loads the input data D = 1011 into Q. In hold mode (mode = 2'b00), the stored data remains unchanged. In shift-right mode (mode = 2'b01), the register shifts its bits to the right and inserts the serial input SR = 0 at the MSB. In shift-left mode (mode = 2'b10), the register shifts its bits to the left and inserts the serial input SL = 1 at the LSB. Finally, parallel load mode loads D = 1100 into Q.

12 a. Moore sequence detector

A Moore machine is a type of finite state machine (FSM) used in digital circuits, where the output depends only on the current state. It works by transitioning between states according to the input received at each clock cycle.

Design Code Testbench Code:

```
`timescale 1ns/1ps

module tb_moore_detector;

    reg clk, reset, din;
    wire detected;

    moore_detector uut (
        .clk(clk),
        .reset(reset),
        .din(din),
        .detected(detected)
    );

    // Clock generation
    initial begin
        clk = 0;
        forever #5 clk = ~clk;
    end

    // Input sequence: 1011011
    initial begin
        reset = 1;
        din = 0;

        repeat (2) @(negedge clk);
        reset = 0;

        @(negedge clk); din = 1;
        @(negedge clk); din = 0;
        @(negedge clk); din = 1;
        @(negedge clk); din = 1;
        @(negedge clk); din = 0;
        @(negedge clk); din = 1;
        @(negedge clk); din = 1;

        @(negedge clk);
        $finish;
    end

    // Generate waveform file
    initial begin
        $dumpfile("moore_wave.vcd");
        $dumpvars(0, tb_moore_detector);
    end

endmodule
```

```
module moore_detector(
    input clk,
    input reset,
    input din,
    output reg detected
);

    parameter S0 = 3'b000,
              S1 = 3'b001,
              S2 = 3'b010,
              S3 = 3'b011,
              S4 = 3'b100;

    reg [2:0] state, next_state;

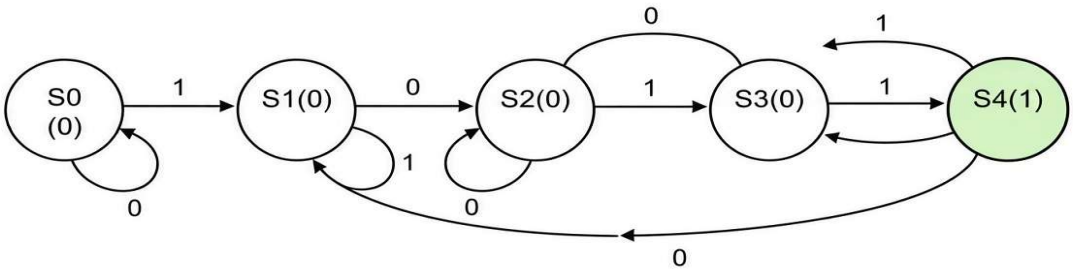
    // State register
    always @(posedge clk or posedge reset) begin
        if (reset)
            state <= S0;
        else
            state <= next_state;
    end

    // Next-state logic
    always @(*) begin
        case (state)
            S0: next_state = din ? S1 : S0;
            S1: next_state = din ? S1 : S2;
            S2: next_state = din ? S3 : S0;
            S3: next_state = din ? S4 : S2;
            S4: next_state = din ? S1 : S2;

            default: next_state = S0;
        endcase
    end

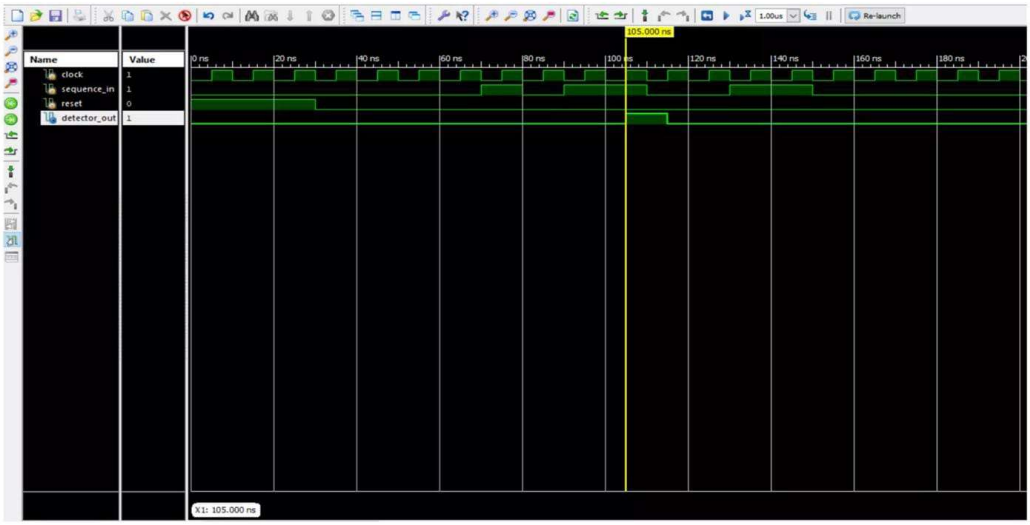
    // Moore output logic
    always @(*) begin
        detected = (state == S4);
    end

endmodule
```



Grok

Output Waveform:



4. Expected output waveform

The input sequence is `1011011`. It contains two overlapping occurrences of `1011`.

Illustrative timing diagram — each column represents one clock cycle after reset.

Signal	1	2	3	4	5	6	7
clk	1	1	1	1	1	1	1
din	1	0	1	1	0	1	1
detected	0	0	0	1	0	0	1

Output Waveform: The waveform shows the clock, reset, input, and output signals during simulation. The clock controls state transitions, and reset initializes the machine. When the required input sequence is detected, the machine enters a specific state and the output becomes 1. Otherwise, the output remains 0, demonstrating the correct operation of the Moore machine.

12 b. Mealy Sequence Detector

A **Mealy machine** is a finite state machine (FSM) in which the output depends on both the present state and the current input. Unlike a Moore machine, a Mealy machine can produce an output immediately when the required input bit arrives, without needing an additional state to indicate detection.

Design Code Testbench Code:

```
`timescale 1ns/1ps

module tb_mealy_detector;

    reg clk, reset, din;
    wire detected;

    mealy_detector uut (
        .clk(clk),
        .reset(reset),
        .din(din),
        .detected(detected)
    );

    initial begin
        clk = 0;
        forever #5 clk = ~clk;
    end

    initial begin
        reset = 1;
        din = 0;

        repeat (2) @(negedge clk);
        reset = 0;

        @(negedge clk); din = 1;
        @(negedge clk); din = 0;
        @(negedge clk); din = 1;
        @(negedge clk); din = 1;
        @(negedge clk); din = 0;
        @(negedge clk); din = 1;
        @(negedge clk); din = 1;

        @(negedge clk);
        $finish;
    end

    initial begin
        $dumpfile("mealy_wave.vcd");
        $dumpvars(0, tb_mealy_detector);
    end

endmodule
```

```
module mealy_detector(
    input clk,
    input reset,
    input din,
    output reg detected
);

    parameter S0 = 2'b00,
              S1 = 2'b01,
              S2 = 2'b10,
              S3 = 2'b11;

    reg [1:0] state, next_state;

    // State register
    always @(posedge clk or posedge reset) begin
        if (reset)
            state <= S0;
        else
            state <= next_state;
    end

    // Next-state and output logic
    always @(*) begin
        next_state = S0;
        detected = 0;

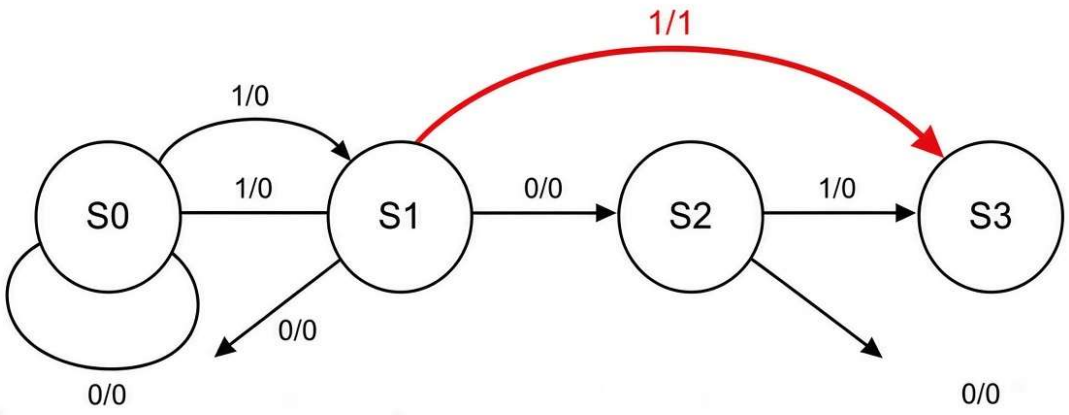
        case (state)
            S0: begin
                if (din)
                    next_state = S1;
                else
                    next_state = S0;
            end

            S1: begin
                if (din)
                    next_state = S1;
                else
                    next_state = S2;end

            S2: begin
                if (din)
                    next_state = S3;
                else
                    next_state = S0;
            end

            S3: begin
                if (din) begin
                    detected = 1;
                    next_state = S1;
                end
                else
                    next_state = S2;
            end

            default: begin
                next_state = S0;
                detected = 0;
            end
        endcase
    end
```



Grok

Output Waveform:



Add these signals to your simulator waveform window in the same order as your screenshot:

Signal	Description
`z`	Output — detects sequence `1011`
`x`	Serial input
`clk`	Clock signal with 10 ns period
`reset`	Initializes the FSM

For the input `1011011`, the output `z` pulses high when the fourth and seventh input bits complete the pattern `1011`.

Output Waveform: The waveform represents the behavior of the Mealy machine during simulation. The clock controls state transitions, reset initializes the machine, and the input determines the next state and output. The output changes according to the current state and input, demonstrating the correct operation of the circuit.